Tool for image annotation in context of modern object detection

Stjepan Prakljačić
TTTech Auto Central and Eastern Europe
Osijek, Croatia
stjepan.prakljacic@tttech-auto.com

Ratko Grbić
Faculty of Electrical Engineering, Computer Science and Information Technology Osijek
Osijek, Croatia
ratko.grbic@ferit.hr

Mario Vranješ
Faculty of Electrical Engineering, Computer Science and Information Technology Osijek
Osijek, Croatia
mario.vranjes@ferit.hr

Marijan Herceg
Faculty of Electrical Engineering, Computer Science and Information Technology Osijek
Osijek, Croatia
marijan.herceg@ferit.hr

2024 Zooming Innovation in Consumer Technologies Conference (ZINC) | 979-8-3503-4915-3/24/$31.00 ©2024 IEEE | DOI: 10.1109/ZINC61849.2024.10579415

***Abstract*—A modern approach to object detection in digital images is based on machine learning, which usually requires a significant amount of annotated data. The annotation process of images involves assigning the class and the location of the objects of interest which can be a time-consuming and tedious task. In this paper, a tool for object annotation in digital images is presented. The tool implements two interfaces: one for project management and one for object annotation in form of bounding boxes. The project management interface allows the users to define the name of the project, choose the storage location, specify the classes of objects of interest, select the project gallery and choose the structured computer format for storing the annotation files. The object annotation interface allows users to create annotations for objects of interest, visualize them, manage the created annotations, and save them in the working annotation dictionary. Users can also configure additional parameters for exporting annotation files to the disk. To further streamline the annotation process, the tool offers automatic detection of objects of interest using the YOLOv5 detector.**

Keywords— computer vision, object detection, annotations

I. INTRODUCTION

Computer vision is an area of artificial intelligence that enables computers and systems to recognize and extract meaningful information from digital images A fundamental problem in computer vision is object detection, where a computer algorithm localizes and classifies objects of interest in digital images. Modern object detectors rely on deep neural networks. To effectively train these networks, a substantial number of annotated images is typically required [1]. These annotations contain information about the position of each object in the image as well as the class or category of the object.

The most common way of annotating objects in a digital image is by drawing a rectangular-shaped enclosure that surrounds an object which is called bounding box. This involves defining the starting point and the height and width of the rectangle around the object. Then a class or category is assigned to that area. The purpose of the bounding box is to define the spatial extent of an object and provide a visual reference for machine learning models trained to detect objects in images.

Efficient and accurate image annotation plays a significant role in successful training of machine learning models for object detection. However, object annotation in digital images is considerably more complex than simple image classification. Using software tools for annotation process not only accelerates the annotation task but also enhances consistency and reduces human error, contributing to the creation of high-quality labeled datasets [2, 3].

This paper introduces a tool designed for annotating objects of interest in digital images. The tool simplifies the process of annotating objects through an intuitive mouse-based interface, allowing users to accurately annotate objects of interest. Additionally, the tool provides user interfaces for project management and object annotation, enabling the creation and administration of user projects, visualization, editing, and deletion of created annotations, and the export of annotation files in three most common structured formats to the disk. By enabling the structure of multiple projects, users are offered the possibility of defining specific requirements on objects of interest whose annotation process can take place on the same set of images at the same time without conflicts during the process. Furthermore, the proposed tool offers the option of generating initial annotations for objects of interest using a deep neural network-based detector. The tool is developed using Python programming language and Qt framework ensuring its portability across different platforms [4].

The paper is organized as follows. Section II provides an overview of the object detection process, data annotation methods, and existing software solutions for annotating objects of interest in digital images. Section III describes the structure of the proposed tool, including the project management interface and the object annotation interface. Section IV illustrates the process of annotating objects in digital images with several examples. Finally, the paper concludes with a summary and suggestions for further enhancements.

II. RELATED WORK

Data annotation is the process of identifying raw data and giving meaningful information to that data, providing context for machine learning models. In computer vision, the process of annotating is reduced to viewing a set of images by a person or team performing the process. Each image is assigned with annotations depending on the problem at hand. In this paper, the focus is on the problem of object detection. Over time, a variety of annotation formats have emerged which support object detection problem, accompanied by the development of various annotation tools.

A. *Annotations formats*

Microsoft Common Objects in Context (MS COCO) is a dataset used in computer vision for object detection, segmentation and classification [1]. The COCO format for storing annotations is a structured way of storing annotations in a single file, using the JSON format based on two structures: a key-value collection and ordered lists of values. Key-value collections are analogous to Python dictionaries, where each key is associated with a corresponding value. The ordered list of values is implemented as a Python list. The image storage file is a standard file in any format, such as Joint Photographic Expert Group (JPG) or Portable Network Graphics (PNG).

The PASCAL VOC format for storing annotations is a structured way of storing annotations, matching each dataset image with a corresponding XML annotation file [5]. The image storage file is a standard file in any format, such as JPG or PNG. Annotation files are structured in XML format and contain bounding boxes of objects and segmentation masks for each object in the image file. The XML file may also include additional information, such as pose, and difficulty flags, which provide more details about the annotated objects.

The YOLO TXT format for storing annotations is a simple way of storing annotations, matching each dataset image with a corresponding text file [6]. Each annotation file contains data, such as the class of the annotated object and the coordinates of its bounding box. Each line in the file corresponds to one object in the image, and when saving to hard disk the file is assigned a name identical to the name of the image file in dataset.

B. *Annotation tools*

In [2] the authors introduced the LabelMe, a web browser-based image annotating tool developed by the Computer Science and Artificial Intelligence Laboratory at MIT. This tool is designed to facilitate the manual annotation of objects in images, enabling the creation of annotated datasets for training computer vision models. The main advantage of the LabelMe tool is accessibility and ease of use. The tool is based on a web browser, which means that no special installation is required, but its use requires a constant Internet connection. The tool allows users to define their own categories, which is especially important in projects with different requirements. One of the main limitations of the tool is the lack of advanced features for collaboration between different users. Additionally, LabelMe does not provide advanced automation tools.

In [3] the authors introduced the VGG Image Annotator (VIA), an open-source image annotation tool developed by the Visual Geometry Group (VGG) at the University of Oxford. The primary goal of VIA is to offer a user-friendly platform for image annotation. The advantage of VIA is its support for circles and ellipses in polygonal annotation. The limitations of VIA is that it offers basic options for project management, but lacks advanced tools for organizing and tracking annotated images in complex projects. Furthermore, VIA does not provide advanced automation tools.

There are other tools like [7] which supports interactive video and image annotation tools. The main feature of this tool is its support for a semi-automatic video and image annotation method, enabling the efficient and accurate generation of ground truth data.

III. PROPOSED TOOL FOR IMAGE ANNOTATION

The proposed tool must enable the creation of configurable projects. This includes defining a project name, selecting a location on the disk, and the ability to define object classes while choosing an image gallery relevant to the project. Once the project is created, the tool should generate a corresponding folder with the project name in the selected disk location. This folder should contain the chosen image gallery and an associated configuration file containing all relevant project parameters for configuring the initial settings of the object annotation interface. The object annotation interface should support image visualization, displaying annotated objects within each image. Users should be able to manipulate images within the project gallery, as well as expand the gallery with new images. Furthermore, the tool should enable users to draw bounding boxes around objects of interest, with the option to adjust annotations by modifying dimensions or positions or to delete them. In scenarios involving automatic object detection, the tool should assess each annotation created. If new object classes or bounding boxes are identified during automatic detection, the tool must ensure their proper incorporation. Throughout the annotation process, users should be able to store individual annotations in the working dictionary. The tool should allow users to export the annotation files to disk in a desired structured format.

In order to achieve the goals set for the tool, the Qt application framework and the Python were chosen for the development and implementation of the tool. Choosing Qt as a tool for development stems from the ability to create complex and intuitive graphical interfaces, ensuring ease of use. Also, it's extensive documentation and community support facilitate efficient development and troubleshooting, while the use of Python libraries, such as OpenCV for image manipulation and PyQt for building graphical interfaces, significantly speeds up the development process. Such approach also ensures tool cross platform.

The functional block diagram of the proposed tool for object annotation in digital images is presented in Fig. 1. The upper part of the diagram represents the configuration and handling of the project parameters, while the lower part represents the proposed solution for the creation, handling and visualization of the created annotations through different modules implemented within the two proposed interfaces.

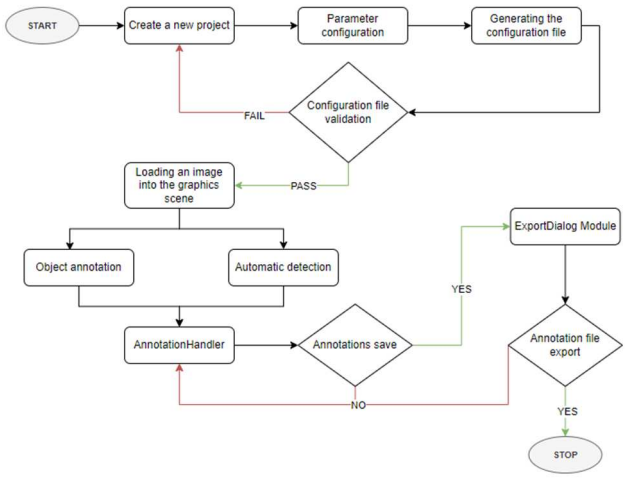


Fig. 1. Functional block diagram of the proposed tool for object annotation in digital images.

A. *Project management interface*

The project management interface, as shown in Fig. 2, provides users with the option to create a new project or access previously created ones. When creating a new project, users must enter a project name, choose a location on the disk, configure object classes of interest, select and store an image gallery, and specify the format for exporting annotation files to disk. The interface consists of the following Qt components:

- *QFrame* - base class in the Qt framework that provides a simple container for other widgets,
- *QWidget* - base class for all UI elements in Qt. It represents a rectangular area on the screen and acts as a container for other widgets,
- *QStackedWidget* - a container that displays only one of its child widgets at a time, stacking them on top of each other,
- *QLabel* – used to display static text or images,
- *QLineEdit* – allows users to enter and edit text in a single line,
- *QPushButton* - implementation is a user-clickable button in the user interface,
- *QListView* – serves to display items in a vertical or horizontal arrangement.

One of the fundamental functionalities of the created interface is to enable users to select the project name and choose the desired project location on the disk. Using the *QStackedWidget* component, the project management interface is divided into several intuitive *QWidget* components. As shown in Fig. 3, the integration of *QLabel* and *QLineEdit* components allows users to enter a project name as well as select the location of the project on disk. The primary task of the tool during this process is to ensure the uniqueness of the project name and the correct project location.

To meet different project needs, the interface provides users with the ability to configure project classes. As shown in Fig. 4, the integration of *QLineEdit* and *QListView* components allows users to enter and customize any number of object classes as needed for their projects. This customization offers users the flexibility to tailor the project to their individual preferences and goals. The primary task of the tool during this process is to ensure the availability of at least one object class of interest.

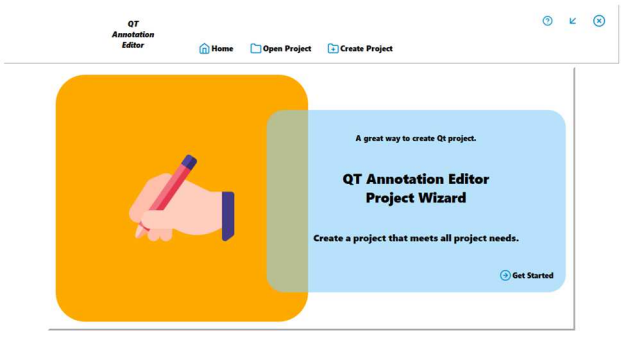


Fig. 2. Project management interface.

In addition, the project management interface allows the user to select a gallery that will serve as a repository for images and files with created annotations during the annotation process. User is allowed to choose a project gallery and a format for exporting annotation files to disk as shown in Fig. 5. The primary task of the tool during this process is to ensure the correctness of the image format within the project gallery and the selection of the format for exporting files with annotations to disk.

B. *Interface for object annotation*

The core functionality of the interface for object annotation in digital images relates to providing user control over the image gallery within the project. The interface should also allow user to easily navigate through the gallery.

Fig. 3. An example of configuring project parameters, entering the project name and selecting the location of the project on disk.

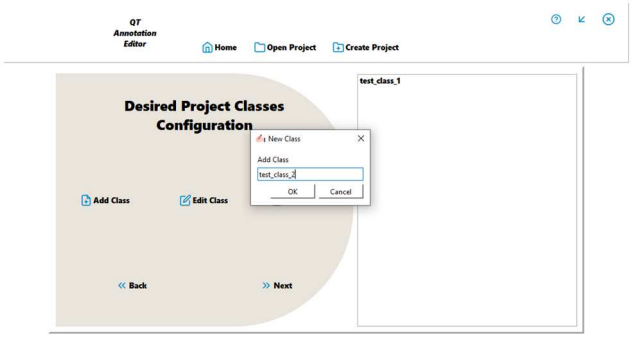


Fig. 4. An example of the configuration of project classes with the display of the user dialog for adding a new instance of the project class.

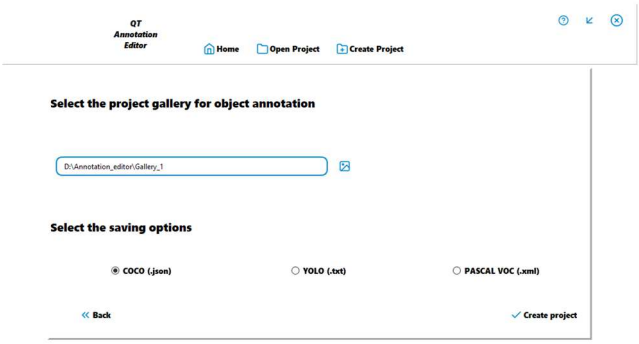


Fig. 5. An example of configuration of project parameters, selection of project gallery and format for exporting a file with annotations to disk.

Furthermore, it should enable the scaling up and down of image dimensions to enhance annotation precision as needed. These functionalities are realized by integrating following Qt components:

- *QGraphicsView*: this component provides a 2D scene view, serving as a viewport where graphics items are displayed and manipulated.
- *QGraphicsScene*: this component is a container class in the Qt framework responsible for holding and managing the graphics items displayed within the *QGraphicsView*.

The functionalities of the proposed interface are realized using the capabilities of the Qt framework known as the toolbar. The toolbar provides users with a set of tools for object annotation and image manipulation. *QActions*, representing actions users can trigger, are implemented within the toolbar. Each *QAction* corresponds to a specific tool or function, such as drawing bounding boxes, storing annotations, or changing the image display dimension. Fig. 6 shows the implementation of the toolbar functionality.

Options „Load Image“ and „Load Video“ allow users to add individual images and videos to the project gallery, ensuring continuous expansion of the dataset. The tool maintains consistency by standardizing the renaming process for newly added images, following the storing practice within the gallery.

The „Draw“ option allow users to create annotations by drawing bounding boxes using the mouse within the *QGraphicsView* component. Pressing the mouse indicates the first coordinate of the bounding box, while dragging changes its height and width. Upon releasing the mouse, the creation process finishes, storing the bounding box coordinates in the annotation attribute. Users can subsequently update these coordinates and assign a class to the annotated object. To facilitate class selection during object annotation, the *ClassDialog* module was developed. After completing the annotation process, user selects the object's class via the dialog box, as illustrated in Fig. 7. By choosing the predefined class set during project setup and clicking the „Add“ button, the object annotation process is completed.

By pressing the toolbar „Delete“ option, users can remove previously created bounding boxes and, thus, the associated annotations. The tool visually presents the image within the *QGraphicsView* after deletion, confirming the successful completion of the process to the user.

To ensure improved visualization of objects of interest during the annotation process, user can enlarge or reduce the image display using the „Zoom In“ or „Zoom Out“ options on the toolbar. Additionally, by using the „Next Image“ or „Previous Image“ options on the toolbar, users can navigate between annotated images displayed in the graphic scene.

By using the „Save“ options on the toolbar, users can save created annotations in the annotations working dictionary for

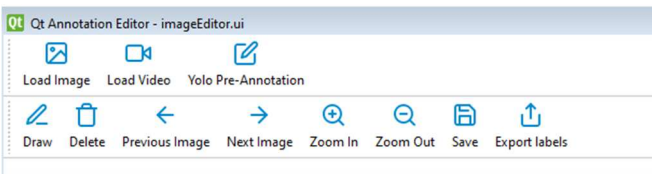


Fig. 6. Toolbar implementation.

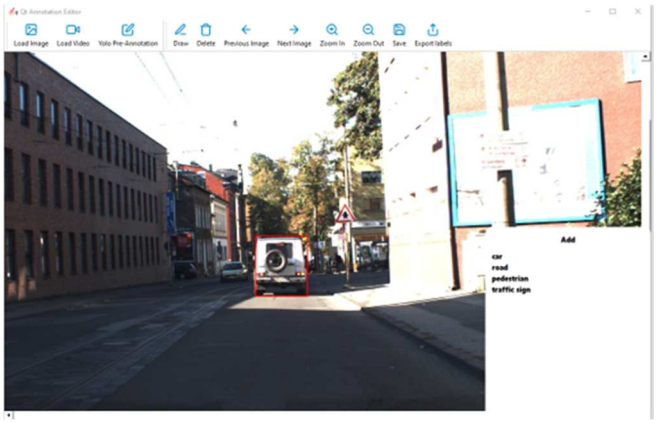


Fig. 7. An example of of project gallery image visualization and usage of the tool module to select the class of annotated object with previously defined classes: "car", "road", "pedestrian", "traffic sign".

each image within the project set. The *AnnotationHandler* module oversees the tool, ensuring the storage of annotations on the displayed image and confirming their persistence in the working dictionary. If an annotation is not present, the tool adds it. Furthermore, when there are changes in annotation dimensions, the tool updates the bounding box values and informs the user of successful storage.

For simplified configuration of parameters for exporting annotation files, the *ExportDialog* module was developed. By selecting the „Export“ option in the toolbar, the interface presents a dialog box, enabling users to modify the predefined export format and location for exporting annotation files to disk. After selecting the desired storage parameters, the tool processes the annotations via the *AnnotationHandler* module and exports the annotation files in the specified format to the chosen location.

C. *Automatic object annotation*

By using the „Yolo Pre-Annotate“ option in the toolbar, users can initiate the initial detection process for objects of interest within displayed image using the YOLOv5 detector. The detector is integrated into the tool via the PyTorch library and automates the annotation process. Basically, for the given image the inference is performed, and a list of detected objects is obtained. Each object is represented by a bounding box with associated class predictions and confidence score. To obtain annotations from these detections, tool utilizes the highest probability of the class for each detection. This process involves analyzing the inference results for each detected object, considering the class predictions and their associated confidence scores. Subsequently, the annotation is determined based on the class with the highest probability. The obtained annotations are appended to the working dictionary, ensuring that the annotation information is systematically recorded and preserved.

IV. Results And Discussion

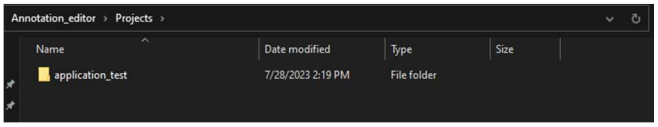
The verification of the proposed tool was performed regarding the functionalities of the project management interface, object annotation interface, and automatic object detection. It's important to note that the verification of results is conducted visually. Verification of the project management interface involved initiating a new project named „application test“. By default, the project location was set within the „Projects“ folder. To validate the process of assigning classes defining objects of interest, „test class 1“ and „test class 2“

were created. The chosen gallery, „Gallery_1," encompasses thirty PNG format images. If a folder is selected that does not contain image files, indicating that it's not intended as the gallery folder, the tool generates an empty folder. This allows users the option to subsequently incorporate images using the toolbar feature within the object annotation interface. The default export format for annotation files is COCO JSON. After the completion of the project creation process, the tool finalizes the creation of the dedicated project folder, encompassing the gallery's contents and a configuration file. In order to ensure the accurate creation of the project, users are promptly notified of any missing or misconfigured parameters. Fig. 8 shows the successful completion of the project creation process, affirming the interface's capability in creating dedicated project folder, encompassing the gallery's contents and a configuration file.

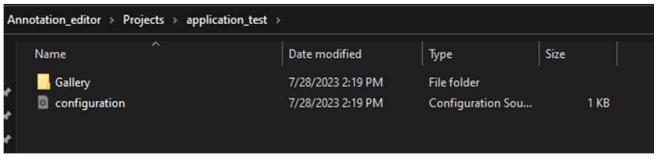
Verification of the Object annotation interface underwent verification within project initiated by the project management interface. This phase focused on verifying the precision of object annotation functionalities, emphasizing the annotation handling, visualization, and export of files containing annotated objects. Fig. 9 shows an example image from the gallery of the „application_test" project, which were annotated using implemented toolbar.

Upon exporting annotations to the desired disk location, one or more files containing annotations in the selected format are obtained. Fig. 10, 11 and 12 demonstrate the export of the annotations to the project gallery location in COCO JSON format, PASCAL VOC XML format and YOLO TXT format, respectively.

The content of the „annotations.json" file is shown in Fig. 13. The content of a single annotation file in XML format is shown in Fig. 14. The content of a single annotation file in YOLO TXT format is shown in Fig. 15.



a)



b)

c)

Fig. 8. a) Verification of the successful creation of the project folder at the selected project location b) verification of successful storage of gallery „Gallery_1" inside the project folder c) display the content of the generated configuration file.

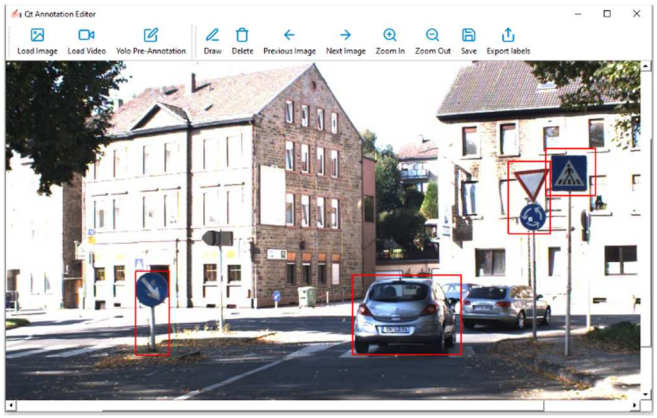


Fig. 9. Display of image „000001" with annotated objects of interest.

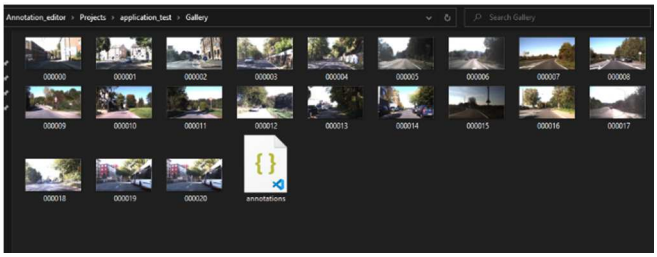


Fig. 10. Visualization of the annotation file at the project gallery location in COCO JSON format.

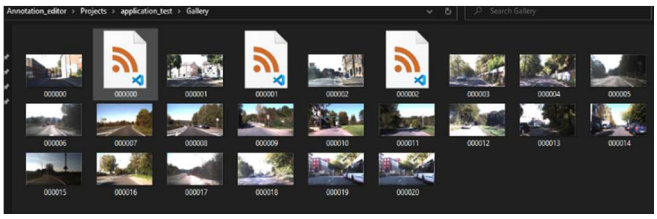


Fig. 11. Visualization of the annotation files at the project gallery location in PASCAL VOC format.

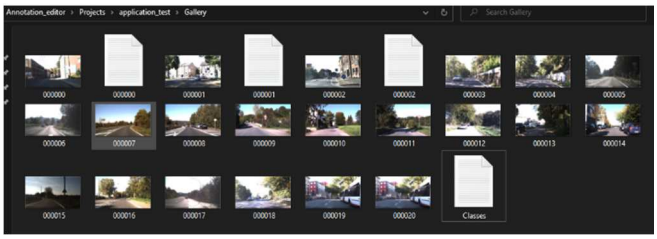


Fig. 12. Visualization of the annotation files at the project gallery location in YOLO TXT format.

For each supported format, the annotations were correctly saved to disk. The verification process was also performed for automatic object detection employing the YOLOv5 detector, aiming to validate the precision in object annotation, visualization, and storage of detected objects within the working dictionary. This process confirmed successful implementation and addition of previously non-existent object classes using automatic detection of objects within the working dictionary.

V. CONCLUSION

The image annotation tool presented in this paper features two interfaces: a project management interface and an object annotation interface. The project management interface allows users to input project configuration parameters, including the project name, selection of the project location on the disk, configuration of project classes, gallery selection, and format for exporting the annotation file to disk. The object annotation interface enables precise annotation of objects by drawing bounding boxes and selecting predefined classes (categories). This interface also supports gallery extension, allowing the incorporation of new images and videos into the existing dataset, along with an additional configuration parameter for exporting the annotation file to disk.

Furthermore, the tool offers various enhancement options. One option is to enable the annotation of objects using polygons. Additionally, the bounding boxes of each created annotation can be color-coded based on the selected class, enhancing the visualization of the variety of annotated objects. To further refine the process of automatic detection, users could have the option to choose detectors of different complexity.

Fig. 13. The content of the „annotations.json“ file.

Fig. 14. The content of the „000000.xml“ file

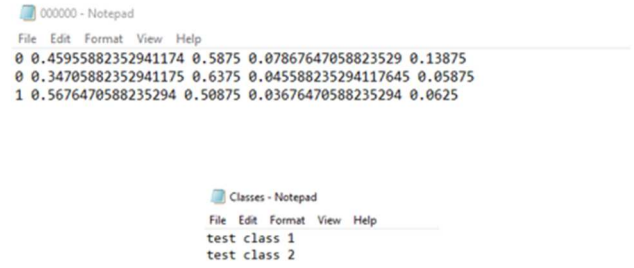


Fig. 15. The content of the „000000.txt“ file.

REFERENCES

[1] T.-Y. Lin et al., „Microsoft COCO: Common Objects in Context,“ in Computer Vision – ECCV 2014, D. Fleet, T. Pajdla, B. Schiele, and T. Tuytelaars, Eds., Cham: Springer International Publishing, 2014.

[2] B. C. Russell, A. Torralba, K. P. Murphy, and W. T. Freeman, „LabelMe: A Database and Web-Based Tool for Image Annotation, “ Int J Comput Vis, vol. 77, no. 1–3, 2008, doi: 10.1007/s11263-007-0090-8.

[3] A. Dutta and A. Zisserman, „The VIA Annotation Software for Images, Audio, and Video,“ in Proceedings of the 27th ACM International Conference on Multimedia, Nice, France: ACM, Oct. 2019, doi: 10.1145/3343031.3350535.

[4] „Qt Documentation | Home.“ Accessed: February 19, 2024. [Online]. Available at: https://doc.qt.io/

[5] M. Everingham, S. M. A. Eslami, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, „The Pascal Visual Object Classes Challenge: A Retrospective,“ Int J Comput Vis, vol. 111, no. 1, Jan. 2015, doi: 10.1007/s11263-014-0733-5.

[6] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, „You Only Look Once: Unified, Real-Time Object Detection.“ arXiv, May 9, 2016. doi: 10.48550/arXiv.1506.02640.

[7] S. Park and C. M. Yang, „Interactive Video Annotation Tool for Generating Ground Truth Information,“ in 2019 Eleventh International Conference on Ubiquitous and Future Networks (ICUFN), Aug. 2019, doi: 10.1109/ICUFN.2019.8806150.